

LogPulse AI - Incident Report

Generated : 2026-06-01 21:50:26 UTC

Trace ID : 1b369628-551c-414a-bfcb-75ac97d20748

1 - Incident Summary

Incident ID : 1b369628
Category : Memory
Severity : SeverityLevel.CRITICAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

21:41:16 ERROR worker-api memory usage critical: 94% of 256Mi limit, GC thrashing

3 - Classification

Category : Memory
Severity : SeverityLevel.CRITICAL
Summary : Worker API application's memory usage critically exceeds limit, causing garbage collection thrashing.
Source : Kubernetes

4 - Diagnosis

Root Cause : The worker-api application's provisioned memory limit of 256Mi is insufficient for its current operational requirements, leading to critical memory usage and garbage collection thrashing.
Confidence : 95%
Reasoning : The raw log directly states 'memory usage critical: 94% of 256Mi limit, GC thrashing', clearly indicating the application is hitting its allocated memory ceiling.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Container OOMKilled Despite Low Reported Memory Usage' (RAG confidence=0.5721). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. SSH to the node running the problematic pod.
- 2. Inspect raw cgroup memory usage for the container: ``cat /sys/fs/cgroup/memory/kubepods/.../memory.usage_in_bytes`` (adjust path). Compare with ``memory.limit_in_bytes``.
- 3. Check ``dmesg -T | grep 'Out of memory`` for the specific process killed.
- 4. Temporarily increase the container's memory ``limit`` to see if the OOM kill ceases.
- 5. Investigate if the application is creating large amounts of shared memory or ``tmpfs`` mounts, which are part of cgroup memory but not always in ``RSS``.
- 6. Consider upgrading the node kernel, ``kubelet``, or container runtime if known bugs related to cgroup accounting exist for the current versions.

Commands:

```
$ SSH to the node running the problematic pod.
$ dmesg -T | grep 'Out of memory'
$ kubectl patch deployment worker-api -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}}}}'
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.SAFE**

Safe Commands (3):

```
[OK] SSH to the node running the problematic pod.
[OK] dmesg -T | grep 'Out of memory'
[OK] kubectl patch deployment worker-api -p
'{"spec":{"template":{"spec":{"containers":[{"name":"worker-api","resources":{"limits":{"memory":"4Gi"}}}}}}}'
```

Flagged Commands (0):

(none)

7 - Operator Decision

Operator rejected -- no execution.

8 - Historical Context

Times Seen : 1
First occurrence -- no prior incidents.

9 - Incident Timeline

```
[18:41:44] INCIDENT_RECEIVED -> Incident received (trace_id=1b369628...)
[18:41:44] CACHE_MISS -> Cache miss -- routing to classifier
[18:41:55] INCIDENT_CLASSIFIED -> Memory / Critical
[18:41:58] RAG_RETRIEVED -> Playbook retrieved (filtered) -- Memory conf=0.57
```

[18:42:03] ROOT_CAUSE_GENERATED -> The worker-api application's provisioned memory limit of 256Mi is insufficient for its current operational requirements,
[18:45:26] SOLUTION_GENERATED -> 6 step(s) from playbook
[18:45:26] RISK_ASSESSED -> safe -- 0 flagged
[18:50:26] WORKFLOW_COMPLETED -> Completed in 521712ms